Documentação de testes Artisan Store API

Apresentação prática da API usando o Postman

Aluno	Bruno Custodio de Castro
Professor	Adriano Baião
Projeto	Artisan Store API
Ferramentas	VS Code, Node.js, Postman e SQLite
Data	22 de outubro de 2026

Registro dos testes feitos localmente, com capturas das respostas exibidas no Postman.

Conteúdo

1 Sobre o projeto 3

2 Preparação e execução da API 4

3 Testes realizados no Postman 5

Teste 01 - Teste de conexão com o servidor . . . 5
Teste 02 - Consulta das categorias . . . 6
Teste 03 - Consulta das situações dos produtos . . . 7
Teste 04 - Listagem geral de produtos . . . 8
Teste 05 - Filtro por categoria e paginação . . . 9
Teste 06 - Cadastro de um usuário . . . 10
Teste 07 - Login de cliente com JWT . . . 11
Teste 08 - Cliente sem permissão para criar produto . . . 12
Teste 09 - Bloqueio de cadastro sem token . . . 13
Teste 10 - Login do administrador . . . 14
Teste 11 - Consulta do perfil do administrador . . . 15
Teste 12 - Listagem de usuários autenticada . . . 16
Teste 13 - Cadastro de um produto (CREATE) . . . 17
Teste 14 - Consulta do produto criado (READ) . . . 18
Teste 15 - Alteração de preço e estoque (UPDATE) . . . 19
Teste 16 - Confirmação dos dados alterados . . . 20
Teste 17 - Busca por nome . . . 21
Teste 18 - Exclusão de produto (DELETE) . . . 22
Teste 19 - Conferência da exclusão . . . 23

4 Resumo do que funcionou 24

5 Conclusão 25

1 Sobre o projeto

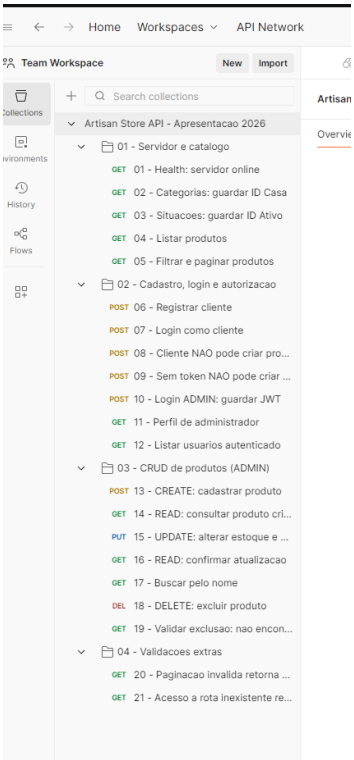
A Artisan Store API é a parte do sistema que recebe pedidos de outros programas e trabalha com os dados de uma loja de produtos artesanais. Não é um site pronto com telas de compra: é o *backend*, onde ficam as rotas, as regras e o acesso ao banco.

O projeto usa Node.js com Express para receber as requisições, Sequelize para trabalhar com o SQLite e JWT para autenticar quem está usando a API. Para testar tudo, usei o Postman, que permite mandar requisições e ver a resposta do servidor.

A ideia da apresentação foi testar o que realmente estava funcionando: consultar produtos e categorias, cadastrar usuário, fazer login, bloquear acessos sem permissão e realizar o CRUD de produtos.

Como organizei os testes

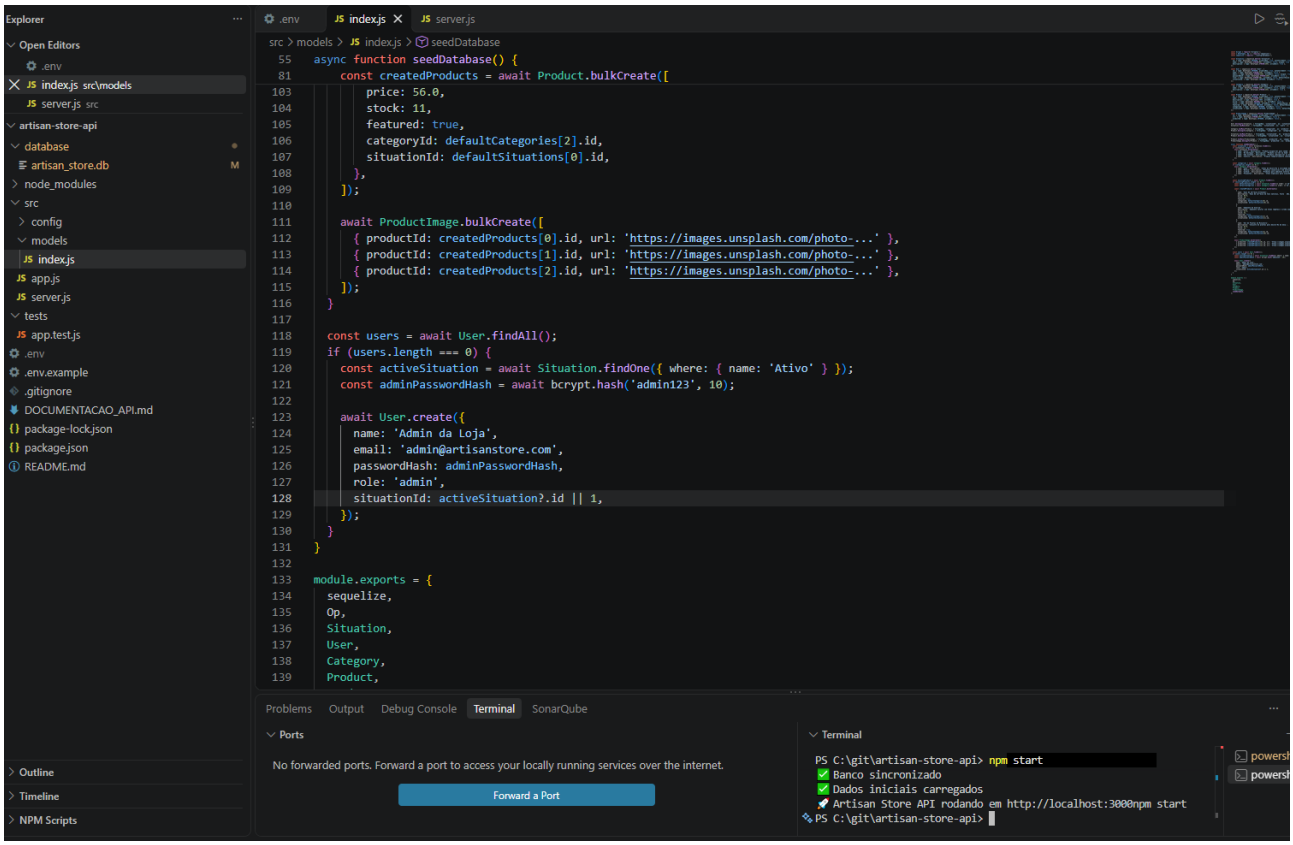
Separei a coleção em quatro pastas: servidor e catálogo; cadastro, login e autorização; CRUD de produtos; e validações extras. A imagem mostra essa organização no Postman.



Sobre os resultados: alguns testes foram feitos para receber respostas de bloqueio. Por isso, HTTP 401, 403 e 404 podem significar teste aprovado, desde que sejam o retorno esperado para aquela situação.

2 Preparação e execução da API

Primeiro abri a pasta do projeto no VS Code e conferi as configurações. No terminal, dentro de `C:\git\artisan-store-api`, iniciei o servidor com `npm start`. A API conecta ao SQLite, sincroniza as tabelas e carrega os dados iniciais.



No começo tive dificuldade para manter a porta 3000 acessível. Depois de ajustar a execução local, confirmei a conexão usando `GET /api/health` no Postman. Os testes seguintes foram feitos com a API respondendo normalmente.

A URL base utilizada foi `http://localhost:3000`. Não usei servidor online: todas as requisições foram feitas na minha máquina. Para testar as rotas que alteram produtos, fiz login como administrador e usei o token JWT retornado.

3 Testes realizados no Postman

Os testes abaixo seguem a mesma ordem da coleção. Cada página mostra o objetivo, o pedido enviado, o resultado que apareceu e a captura usada como comprovação.

Teste 01 - Teste de conexão com o servidor

APROVADO 12 ms / testes 2/2

Método GET
Endpoint `/api/health`
Retorno 200 OK

O que eu fiz. Comecei conferindo se o servidor estava respondendo. Abri a rota de saúde da API e mandei a requisição pelo Postman.

O que aconteceu. Recebi o status 200 OK e a mensagem de que a Artisan Store API estava funcionando. Isso serviu para conferir a conexão antes dos outros testes.

O que esse teste mostra. Antes de testar cadastro ou banco, confirmei se a API estava online.

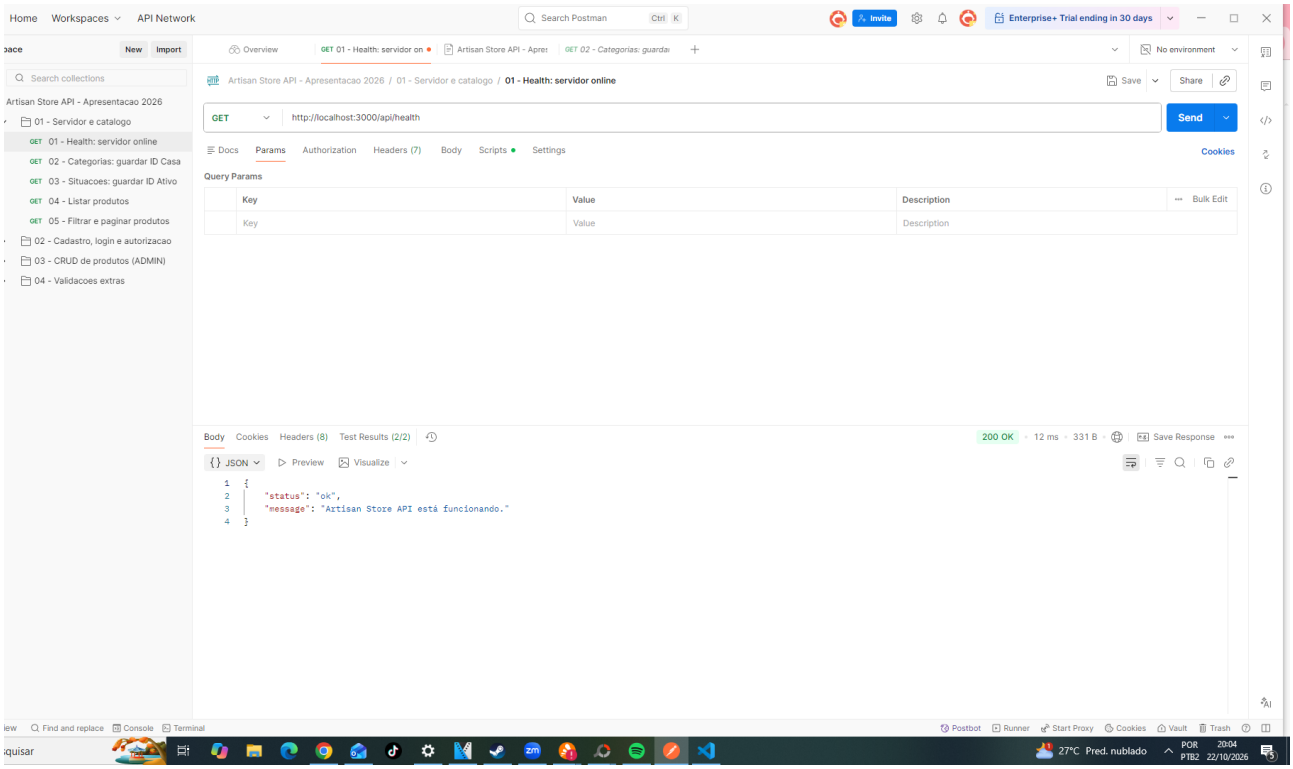


Figura 1: captura do teste 01 no Postman.

Teste 02 - Consulta das categorias

APROVADO 5 ms / testes 2/2

Método GET
Endpoint `/api/categories`
Retorno 200 OK

O que eu fiz. Pedi para a API listar as categorias cadastradas. Usei uma requisição GET, sem enviar dados no corpo.

O que aconteceu. A resposta veio em JSON e mostrou categorias como Casa, Beleza e Jardim. A categoria Casa apareceu com ID 1, que depois usei no cadastro do produto.

O que esse teste mostra. Usei esse teste para descobrir as categorias existentes e aproveitar seus IDs nos próximos pedidos.

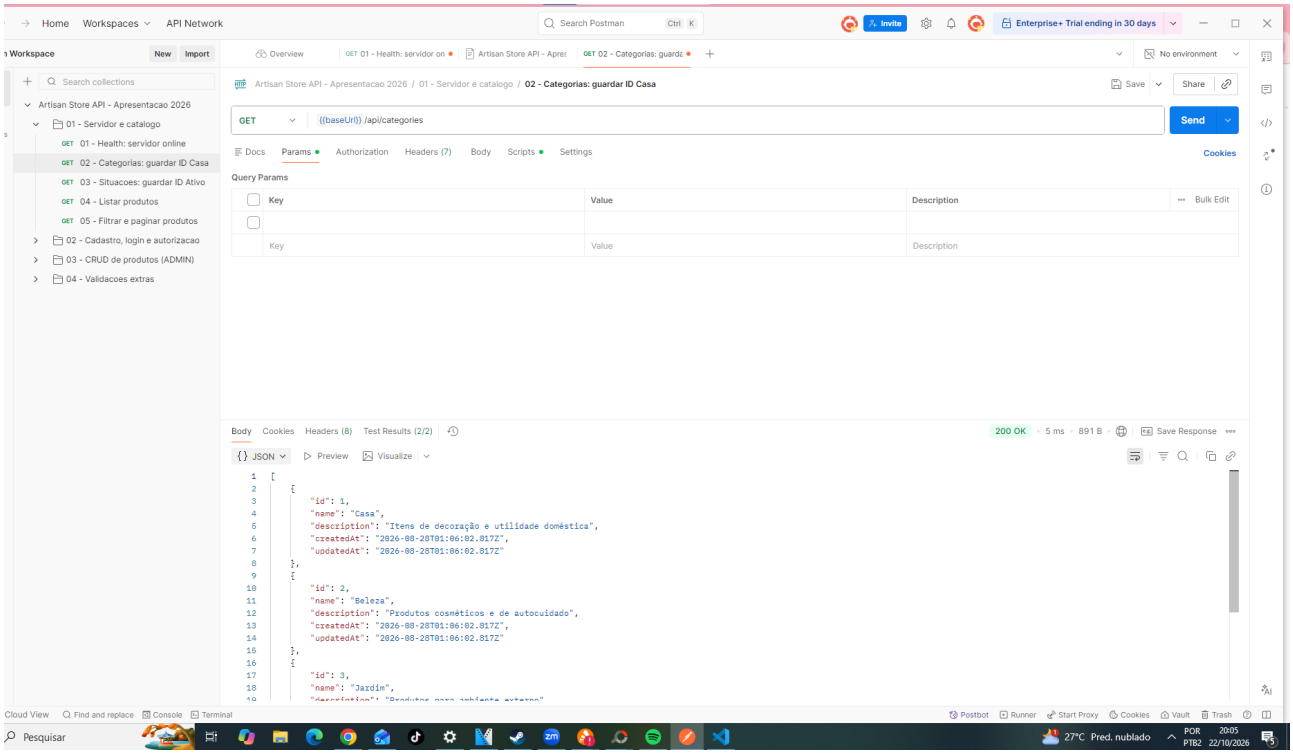


Figura 2: captura do teste 02 no Postman.

Teste 03 - Consulta das situações dos produtos

APROVADO 5 ms / testes 2/2

Método GET
Endpoint `/api/situations`
Retorno 200 OK

O que eu fiz. Consultei as situações que podem ser associadas aos produtos. A requisição também foi GET.

O que aconteceu. O retorno trouxe Ativo, Em destaque e Sem estoque. A situação Ativo apareceu com ID 1 e foi usada no produto criado mais adiante.

O que esse teste mostra. Aqui confirmei que a API consegue consultar os estados definidos para os produtos.

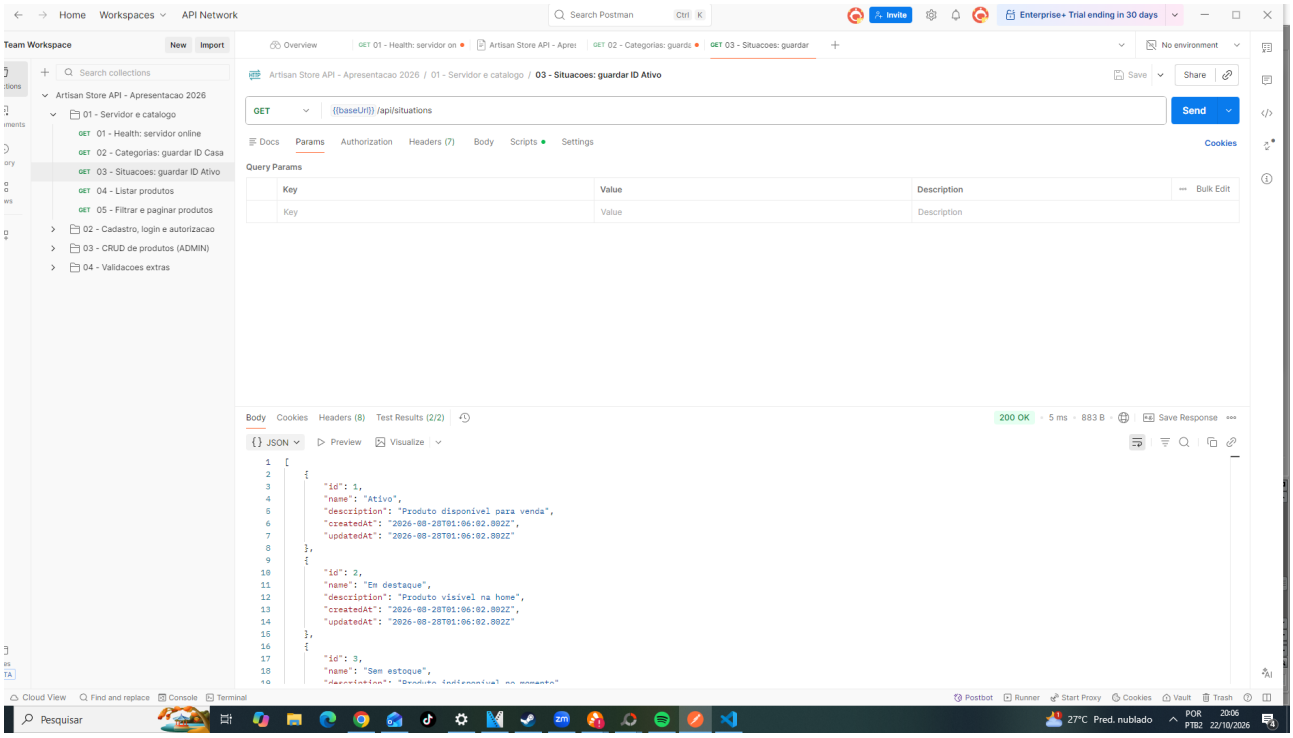


Figura 3: captura do teste 03 no Postman.

Teste 04 - Listagem geral de produtos

APROVADO 14 ms / testes 2/2

Método GET
Endpoint `/api/products`
Retorno 200 OK

O que eu fiz. Pedi uma lista dos produtos já existentes na loja antes de fazer qualquer novo cadastro.

O que aconteceu. A API retornou os registros com nome, descrição, preço, quantidade em estoque, categoria e situação. Um dos itens era o Vaso de Cerâmica Artesanal, com preço de R$ 89,90 e estoque de 18 unidades.

O que esse teste mostra. A intenção foi mostrar que o catálogo é consultado diretamente pela API.

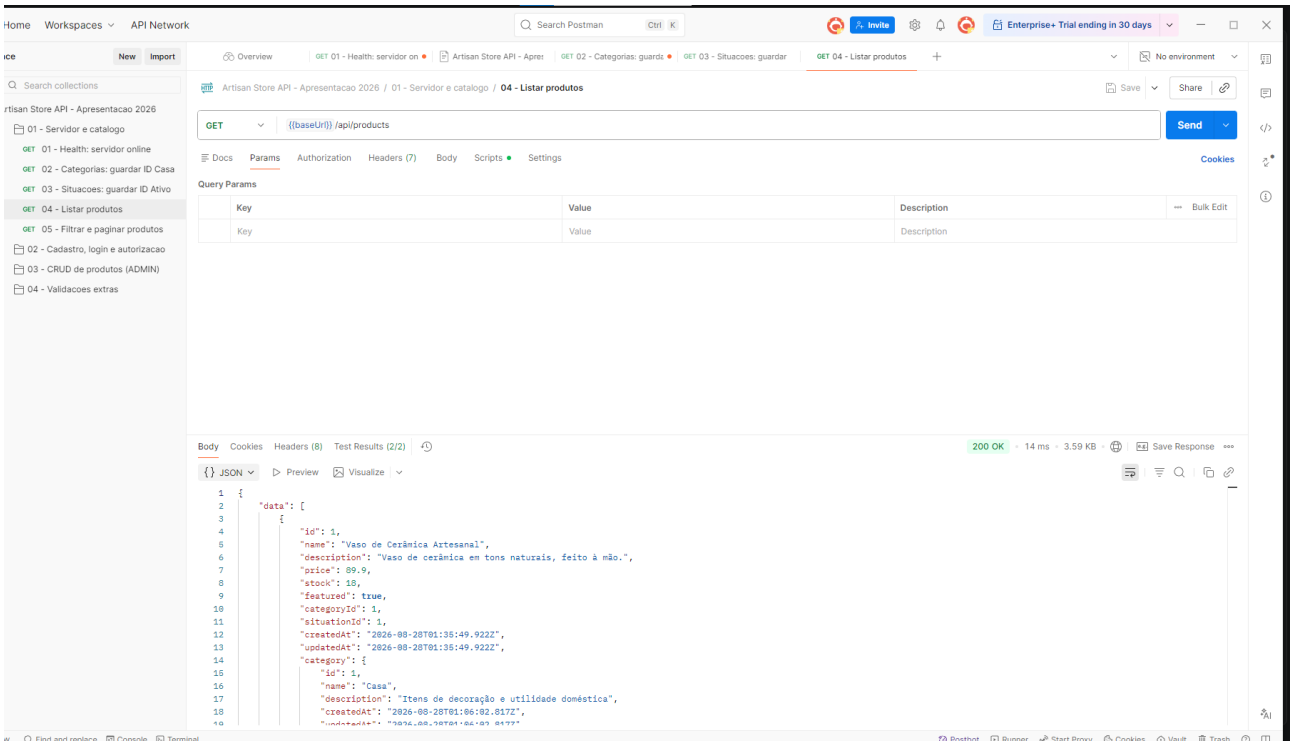


Figura 4: captura do teste 04 no Postman.

Teste 05 - Filtro por categoria e paginação

APROVADO 9 ms / testes 2/2

Método GET

Endpoint `/api/products?categoryId=1&page=1&limit=5`

Retorno 200 OK

O que eu fiz. Apliquei um filtro para ver produtos da categoria Casa. Também informei página 1 e limite de cinco produtos.

O que aconteceu. A requisição retornou o resultado filtrado, incluindo o Vaso de Cerâmica Artesanal. Assim vi que a API aceita parâmetros na URL para controlar uma consulta.

O que esse teste mostra. Esse recurso evita buscar tudo de uma vez quando a loja tem muitos produtos.

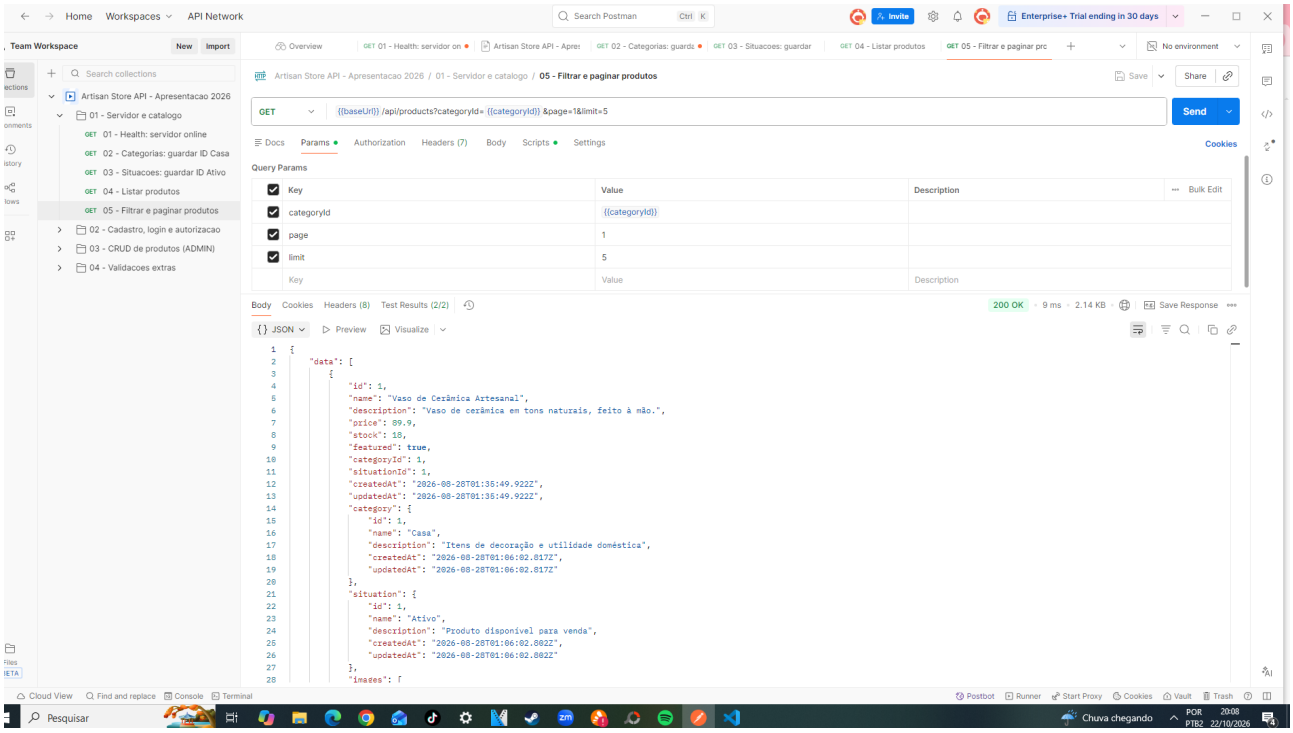


Figura 5: captura do teste 05 no Postman.

Teste 06 - Cadastro de um usuário

APROVADO 95 ms / testes 2/2

Método POST
Endpoint `/api/auth/register`
Retorno 201 Created

O que eu fiz. Mandei nome, e-mail e senha no JSON para criar uma conta de cliente. No corpo da requisição também coloquei o perfil admin para testar o controle de acesso.

O que aconteceu. A API criou o usuário com ID 15 e retornou o perfil customer, não admin. Ou seja, o cadastro público não aceitou transformar um cliente em administrador.

O que esse teste mostra. Foi um cadastro de verdade, mas sem liberar privilégios administrativos só porque foram pedidos no JSON.

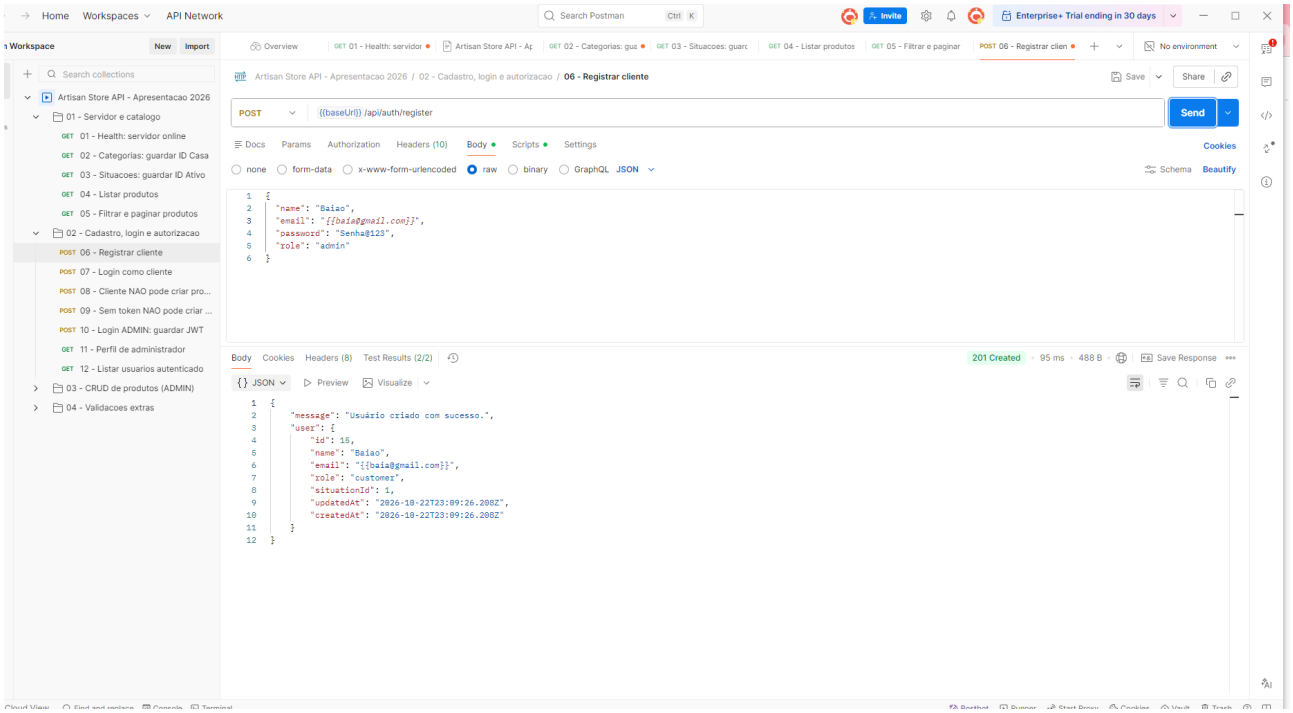


Figura 6: captura do teste 06 no Postman.

Teste 07 - Login de cliente com JWT

APROVADO 62 ms / testes 2/2

Método POST
Endpoint `/api/auth/login`
Retorno 200 OK

O que eu fiz. Usei os dados da conta criada no teste anterior para fazer login pela rota de autenticação.

O que aconteceu. Recebi a mensagem de login realizado com sucesso, os dados do usuário e um token JWT. Na resposta, o perfil do usuário continuou sendo customer.

O que esse teste mostra. O token identifica a sessão nas rotas protegidas, mas o perfil ainda limita o que a pessoa pode fazer.

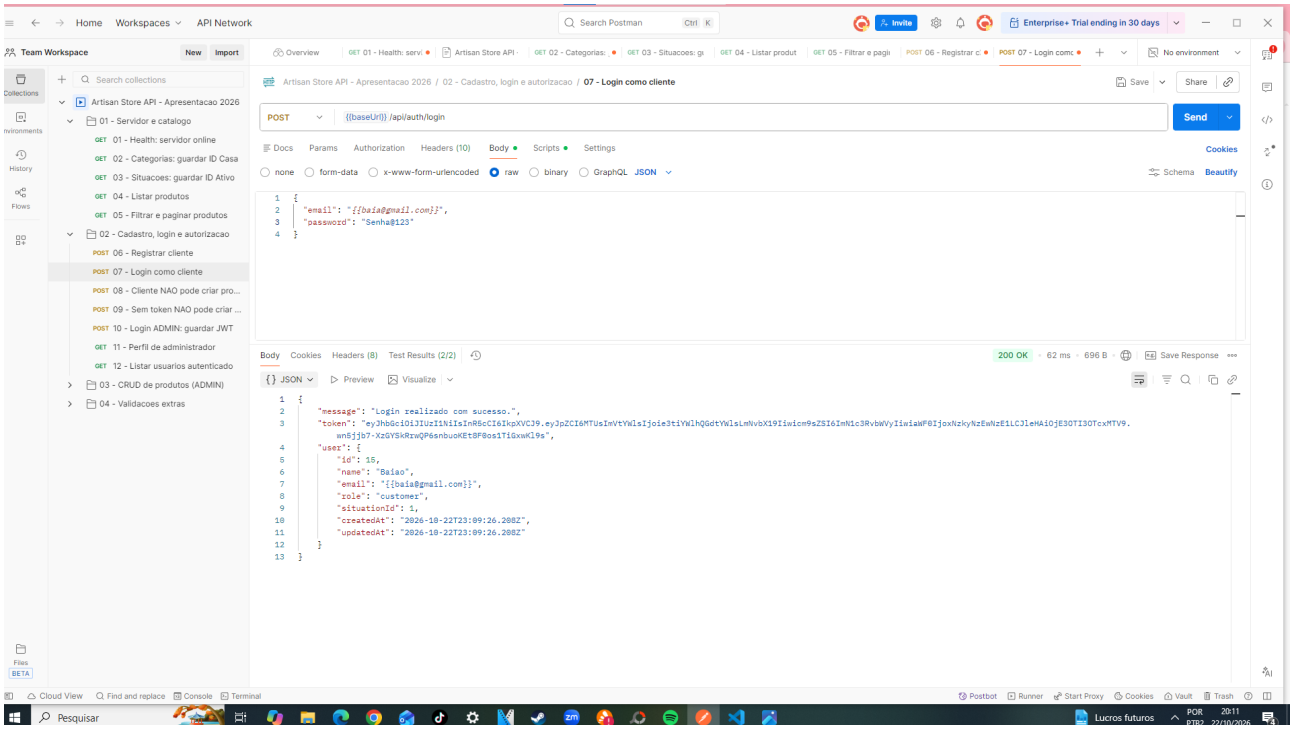


Figura 7: captura do teste 07 no Postman.

Teste 08 - Cliente sem permissão para criar produto

APROVADO - retorno esperado 5 ms / testes 1/1

Método POST
Endpoint `/api/products`
Retorno 403 Forbidden

O que eu fiz. Tentei cadastrar um produto usando a autenticação de um usuário comum, que tinha feito login corretamente.

O que aconteceu. A API negou a operação com a mensagem de acesso permitido apenas para administradores. O HTTP 403 era o resultado esperado, por isso o teste passou.

O que esse teste mostra. Estar logado não significa ter autorização para fazer qualquer alteração.

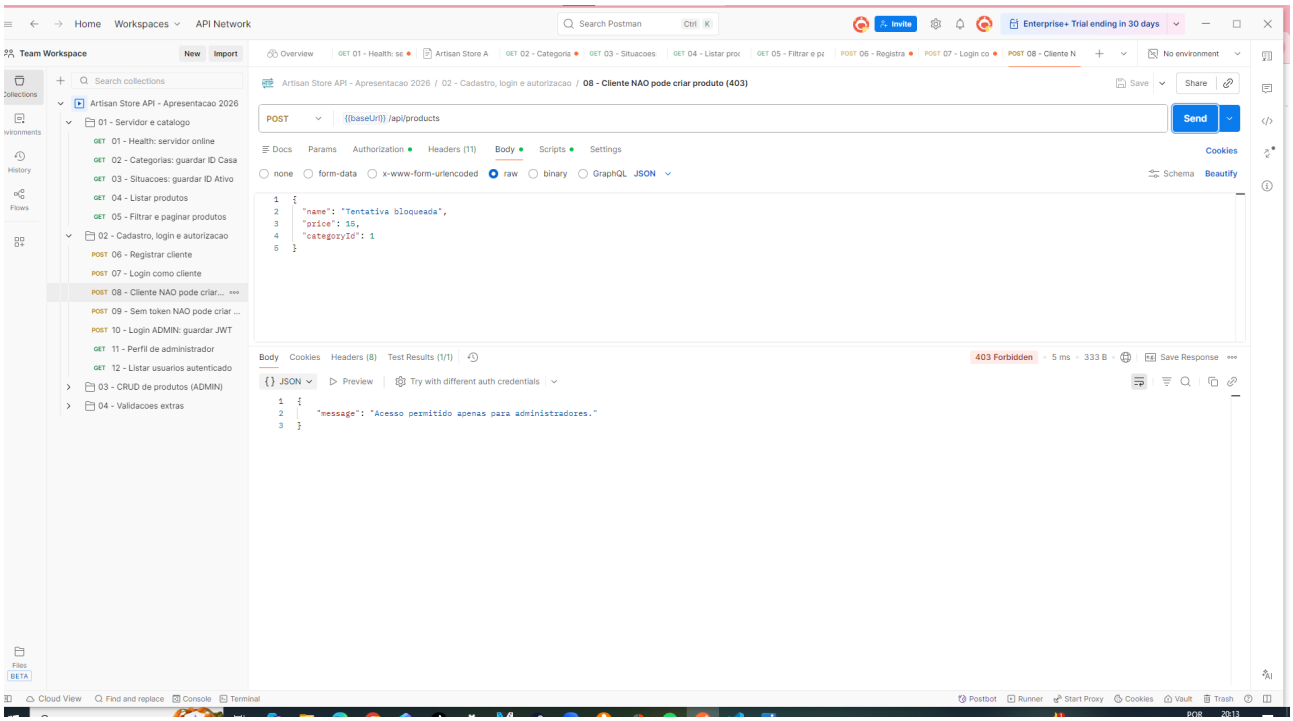


Figura 8: captura do teste 08 no Postman.

Teste 09 - Bloqueio de cadastro sem token

APROVADO - retorno esperado 3 ms / testes 1/1

Método POST
Endpoint `/api/products`
Retorno 401 Unauthorized
O que eu fiz. Tentei enviar um cadastro de produto sem usar token de autenticação.

O que aconteceu. A resposta foi Token de autenticação obrigatório, com HTTP 401. Era exatamente o comportamento esperado para uma rota protegida.

O que esse teste mostra. O 401 mostra ausência de autenticação; no teste 08, o 403 mostrou falta de permissão.

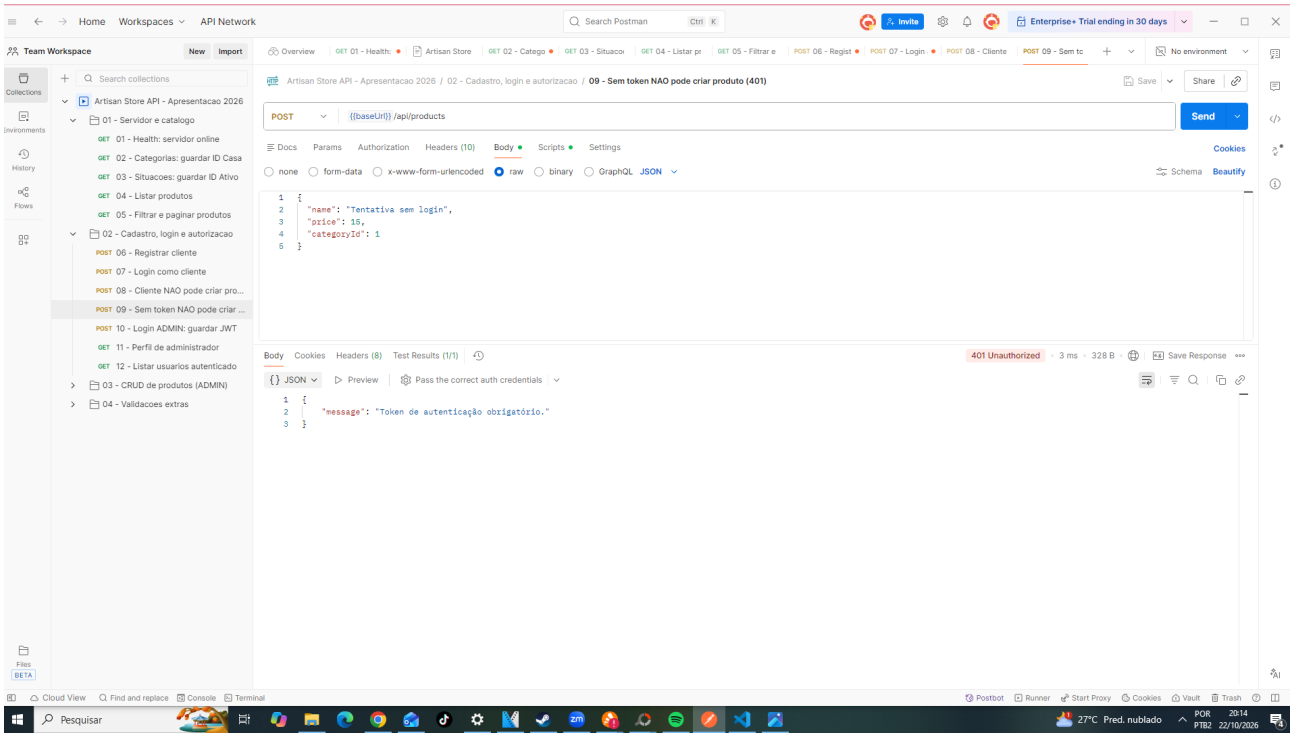


Figura 9: captura do teste 09 no Postman.

Teste 10 - Login do administrador

APROVADO 59 ms / testes 3/3

Método POST
Endpoint `/api/auth/login`
Retorno 200 OK
O que eu fiz. Fiz login com a conta de administrador preparada para os testes da API.

O que aconteceu. A resposta retornou o perfil admin, ID 1 e um token JWT. Com isso, foi possível executar as ações de cadastro, edição e exclusão de produtos.

O que esse teste mostra. Aqui demonstrei a diferença entre o perfil de cliente e o perfil de administrador.

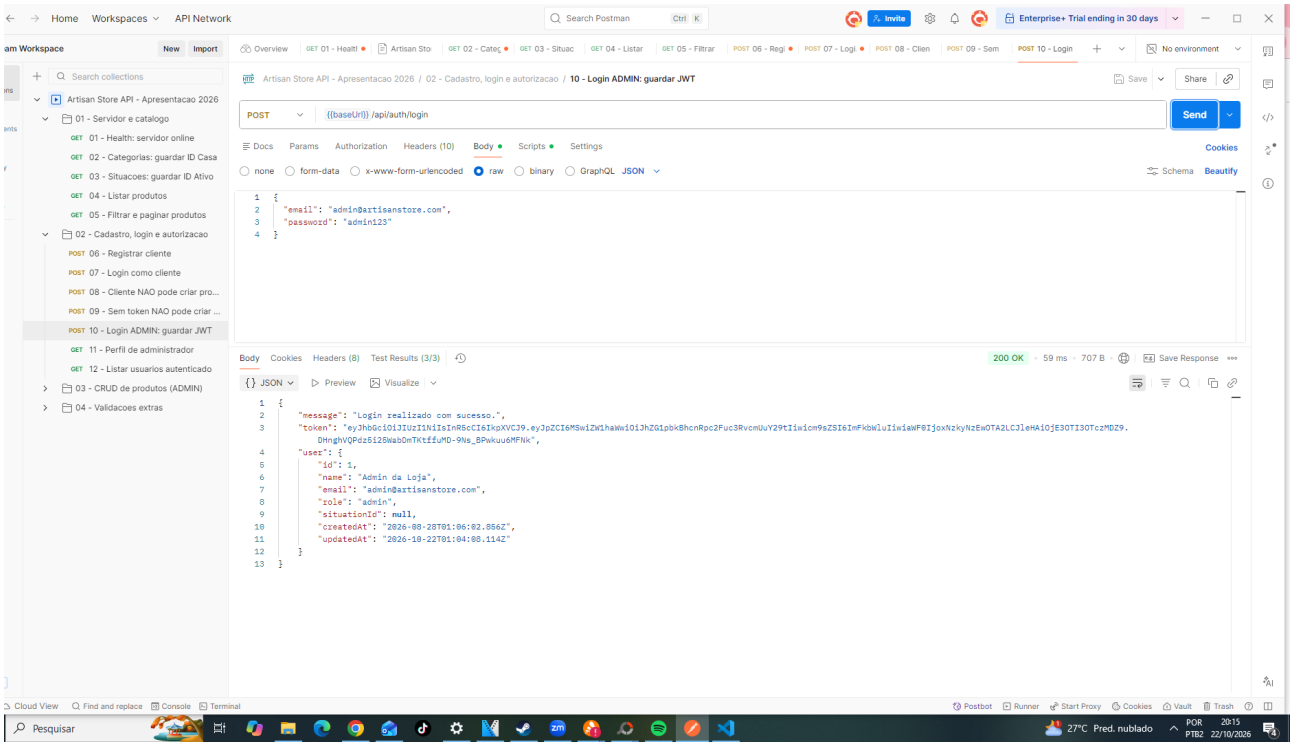


Figura 10: captura do teste 10 no Postman.

Teste 11 - Consulta do perfil do administrador

APROVADO 5 ms / testes 2/2

Método GET
Endpoint `/api/profile`
Retorno 200 OK
O que eu fiz. Depois do login, usei o token administrativo para consultar a rota de perfil.

O que aconteceu. A resposta mostrou o ID 1, o nome Admin da Loja e o perfil admin. Não apareceu hash de senha no retorno.

O que esse teste mostra. O servidor identificou a conta autenticada a partir do token recebido.

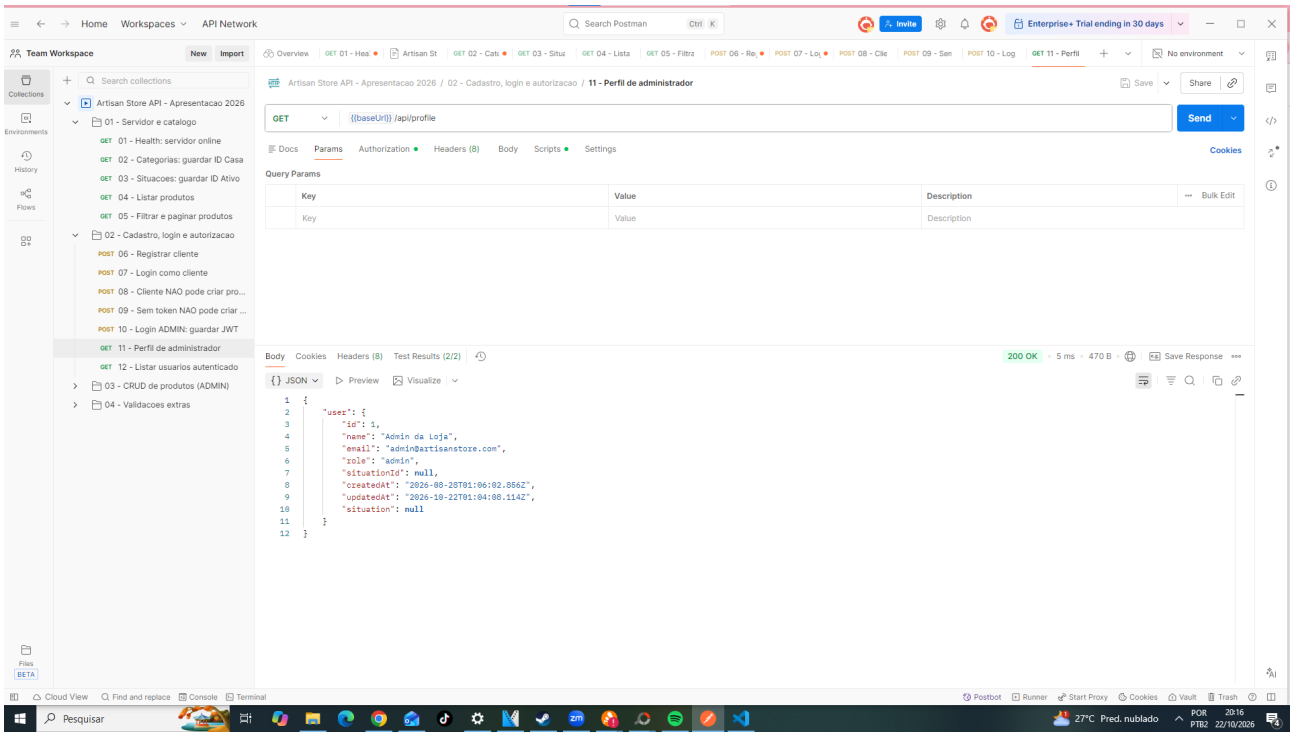


Figura 11: captura do teste 11 no Postman.

Teste 12 - Listagem de usuários autenticada

APROVADO 7 ms / testes 1/1

Método GET
Endpoint `/api/users`
Retorno 200 OK

O que eu fiz. Usei a rota de usuários autenticado como administrador para ver os cadastros existentes.

O que aconteceu. A API retornou uma lista em JSON. Nela apareceu também o usuário criado no teste 06, com ID 15 e perfil customer.

O que esse teste mostra. Foi uma forma de conferir que o cadastro anterior continuava salvo no SQLite.

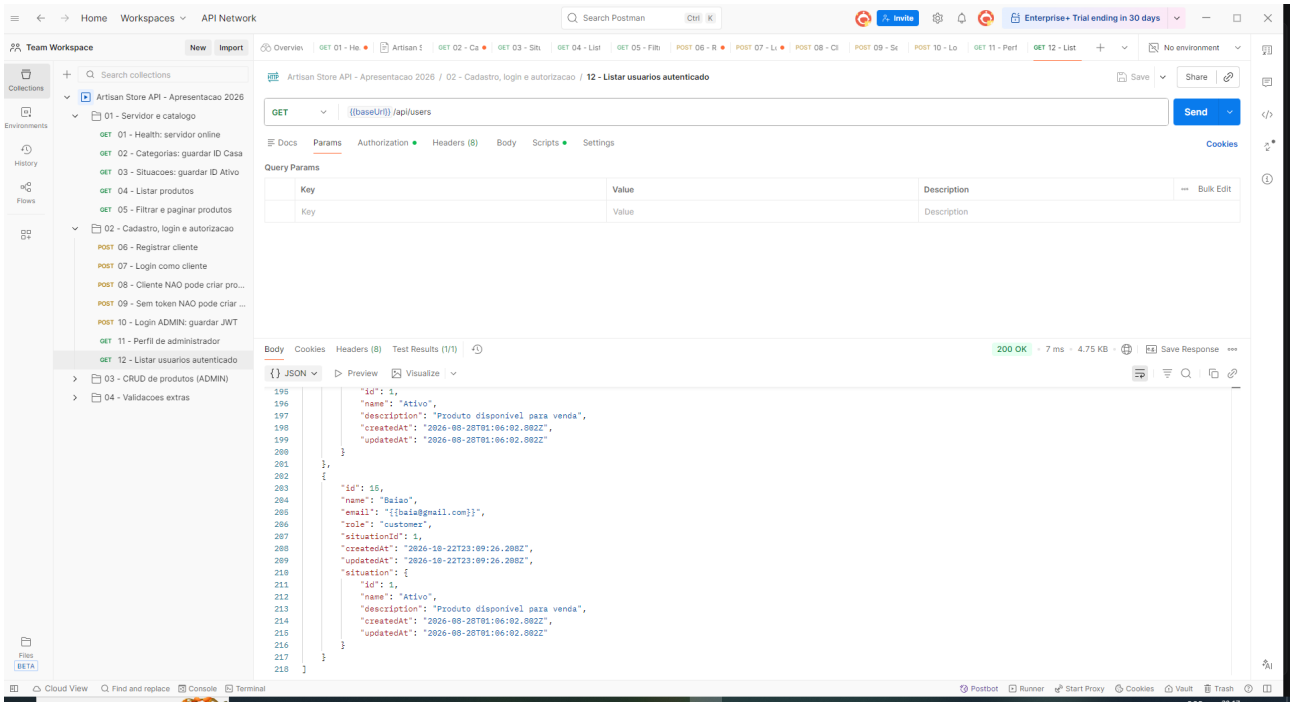


Figura 12: captura do teste 12 no Postman.

Teste 13 - Cadastro de um produto (CREATE)

APROVADO 11 ms / testes 2/2

Método POST
Endpoint `/api/products`
Retorno 201 Created

O que eu fiz. Com o administrador autenticado, cadastrei o produto camisa do baiao.

Enviei nome, descrição, preço, estoque, destaque, categoria e situação no corpo JSON.

O que aconteceu. A API respondeu Produto cadastrado com sucesso e gerou o ID 6. O produto entrou com preço de R$ 9,99 e uma unidade no estoque.

O que esse teste mostra. Essa foi a etapa CREATE: inserir um novo registro no banco.

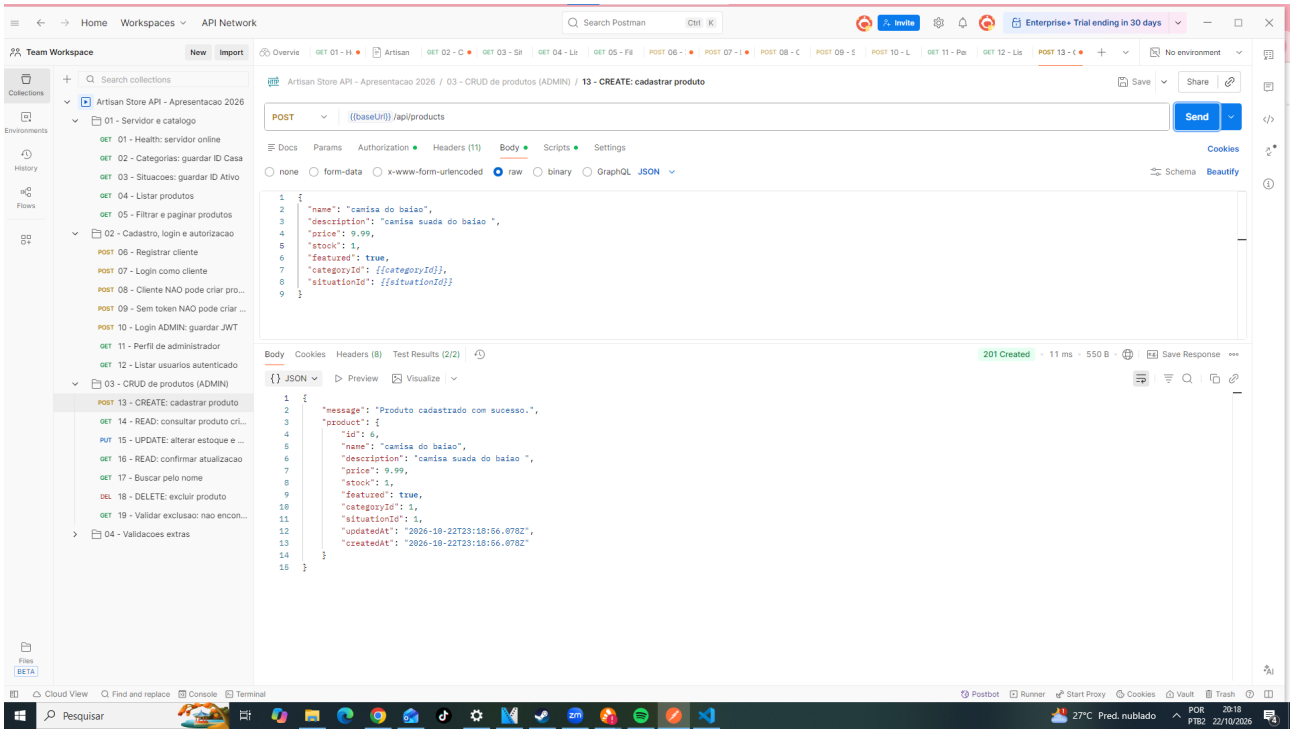


Figura 13: captura do teste 13 no Postman.

Teste 14 - Consulta do produto criado (READ)

APROVADO 6 ms / testes 2/2

Método GET
Endpoint `/api/products/6`
Retorno 200 OK
O que eu fiz. Consultei o produto recém-criado usando seu identificador, 6.

O que aconteceu. O retorno mostrou o mesmo nome, preço de R$ 9,99, estoque de 1 unidade, categoria Casa e situação Ativo.

O que esse teste mostra. Isso confirmou que o produto podia ser recuperado depois do cadastro.

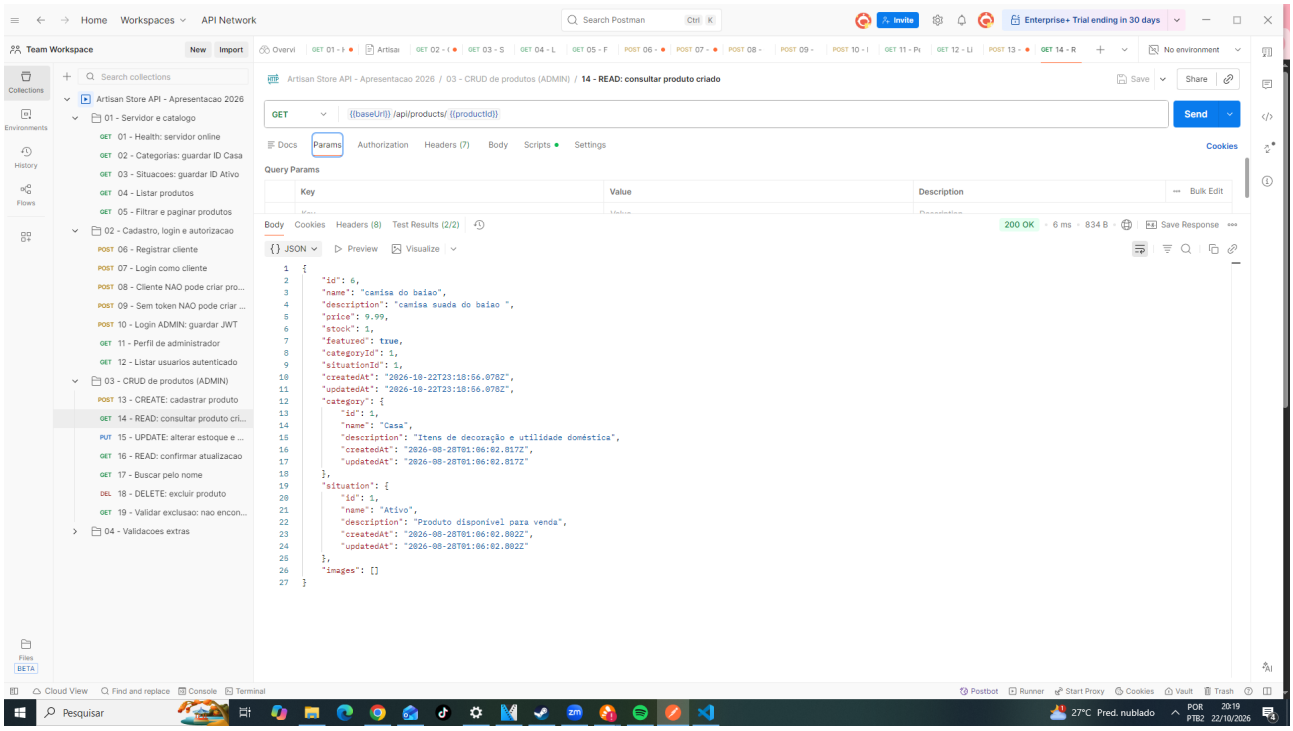


Figura 14: captura do teste 14 no Postman.

Teste 15 - Alteração de preço e estoque (UPDATE)

APROVADO 14 ms / testes 2/2

Método PUT
Endpoint `/api/products/6`
Retorno 200 OK

O que eu fiz. Enviei uma requisição PUT para mudar os dados do produto 6, usando o token de administrador.

O que aconteceu. O preço passou de R$ 9,99 para R$ 79,90, o estoque passou de 1 para 25 unidades e o campo de destaque ficou false.

O que esse teste mostra. Essa foi a etapa UPDATE: atualizar um registro que já existia.

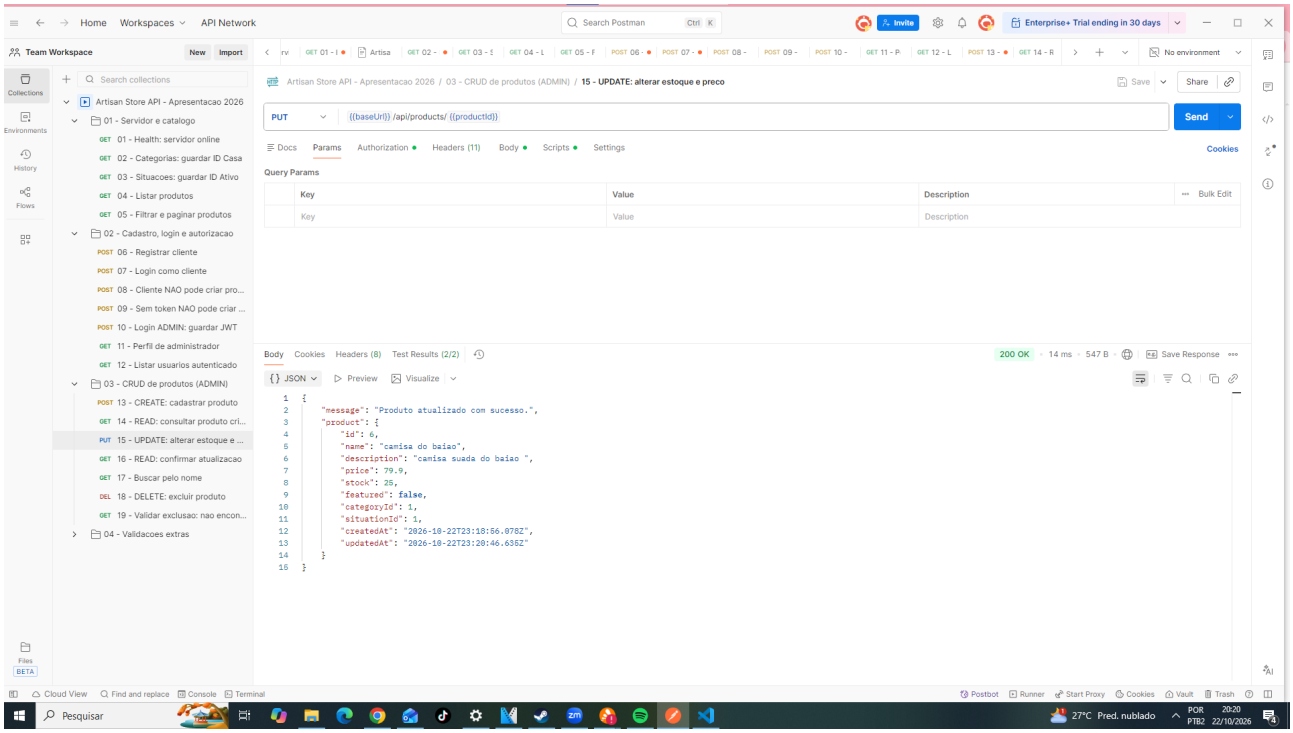


Figura 15: captura do teste 15 no Postman.

Teste 16 - Confirmação dos dados alterados

APROVADO 6 ms / testes 2/2

Método GET
Endpoint `/api/products/6`
Retorno 200 OK

O que eu fiz. Depois do PUT, fiz outra consulta GET do mesmo ID, sem mandar os dados do produto novamente.

O que aconteceu. A resposta já mostrou preço de R$ 79,90, estoque de 25 unidades e destaque desativado. Isso confirmou que a atualização ficou salva.

O que esse teste mostra. Não foi só uma mensagem de sucesso: os novos valores apareceram numa consulta separada.

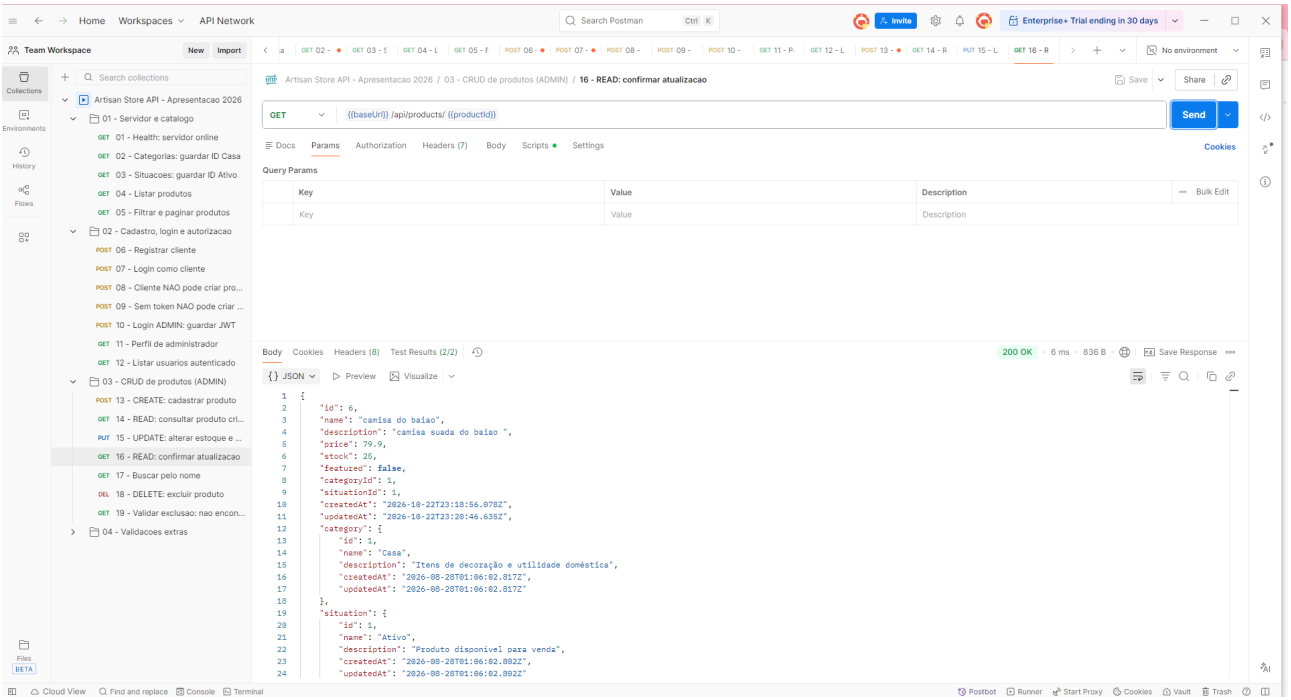


Figura 16: captura do teste 16 no Postman.

Teste 17 - Busca por nome

APROVADO 7 ms / testes 2/2

Método GET

Endpoint `/api/products?search=baiao&page=1&limit=10`

Retorno 200 OK

O que eu fiz. Busquei produtos usando a palavra baiao como termo de pesquisa. Também mantive os parâmetros de paginação.

O que aconteceu. A API encontrou o item camisa do baiao, ID 6, já com o preço atualizado para R$ 79,90 e estoque de 25 unidades.

O que esse teste mostra. Essa consulta mostrou que é possível localizar um produto pelo nome sem conhecer seu ID.

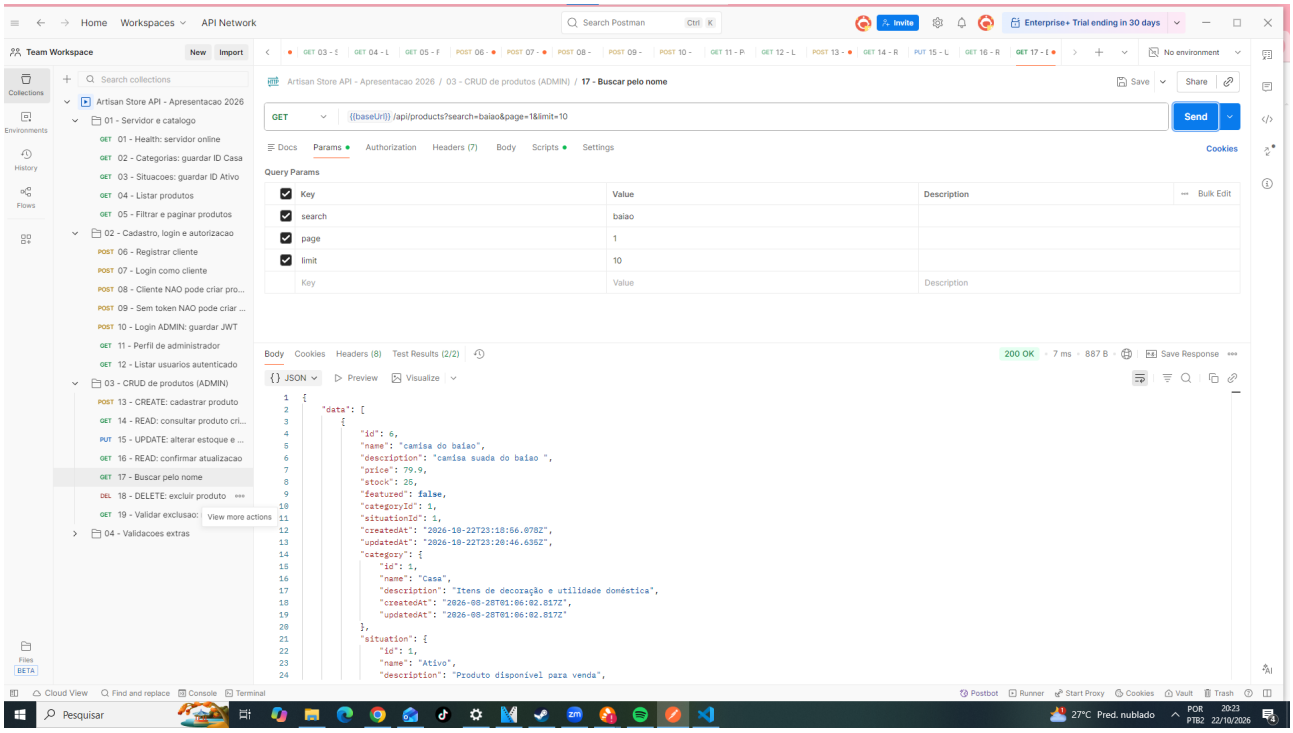


Figura 17: captura do teste 17 no Postman.

Teste 18 - Exclusão de produto (DELETE)

APROVADO 11 ms / testes 1/1

Método DELETE
Endpoint `/api/products/6`
Retorno 200 OK

O que eu fiz. Enviei um DELETE para o produto de ID 6 com a autenticação de administrador.

O que aconteceu. A API respondeu Produto removido com sucesso. O print mostra um parâmetro extra baiao na URL, mas ele não era necessário para excluir pelo ID.

O que esse teste mostra. Essa foi a etapa DELETE do CRUD.

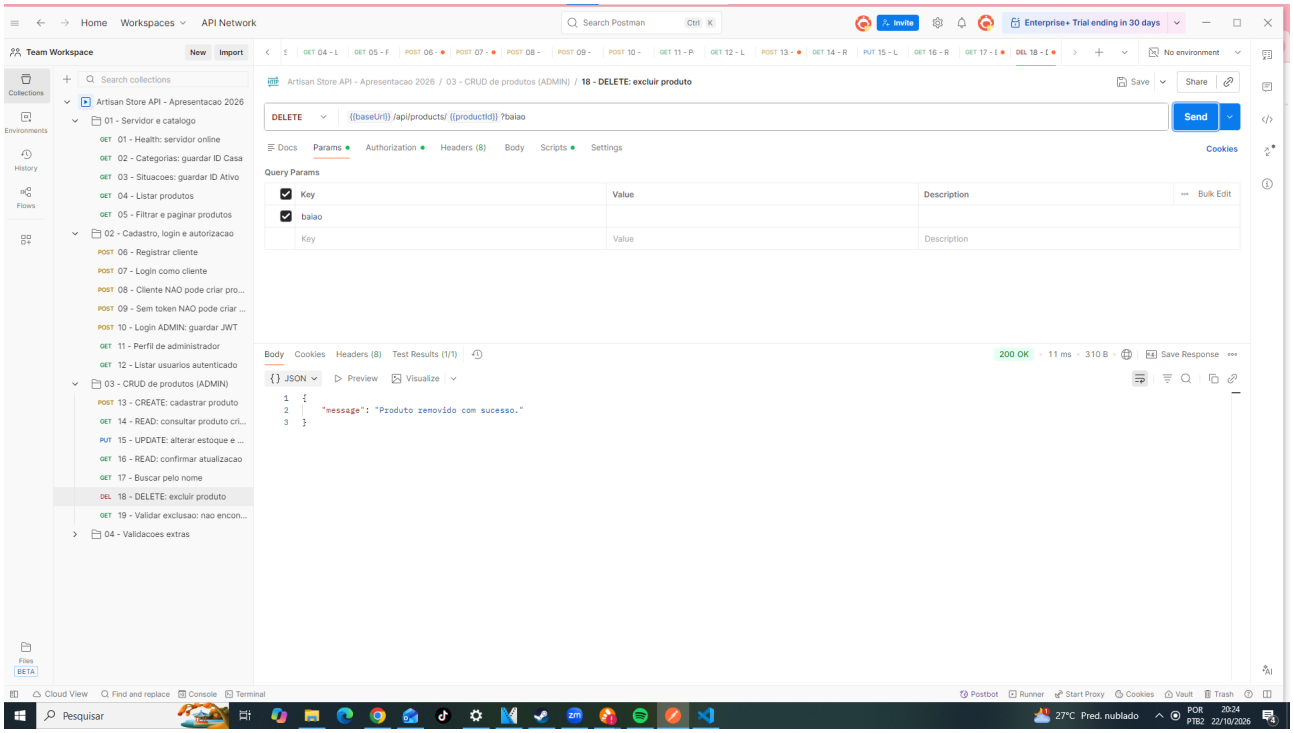


Figura 18: captura do teste 18 no Postman.

Teste 19 - Conferência da exclusão

APROVADO - retorno esperado 5 ms / testes 1/1

Método GET
Endpoint `/api/products/6`
Retorno 404 Not Found
O que eu fiz. Após apagar o produto, tentei consultá-lo novamente pelo mesmo ID.

O que aconteceu. A API retornou Produto não encontrado, com HTTP 404. Nesse caso, 404 é acerto: o produto havia sido apagado e não deveria mais aparecer.

O que esse teste mostra. Com essa consulta, concluí a demonstração de CREATE, READ, UPDATE e DELETE.

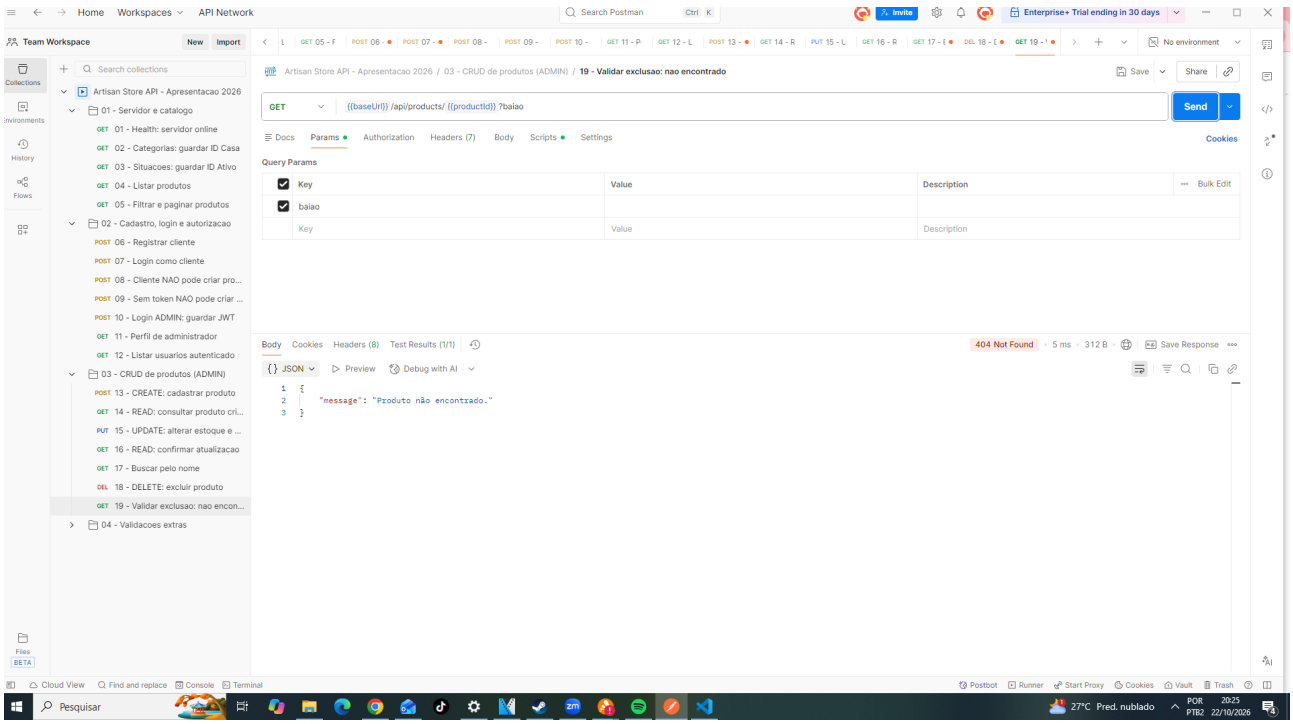


Figura 19: captura do teste 19 no Postman.

4 Resumo do que funcionou

No fim, consegui testar a parte principal da API no Postman. A tabela abaixo resume o caminho feito, do servidor online até a exclusão de um produto de teste.

Parte do sistema	Testes	Resultado
Servidor, categorias, situações e catálogo	01 a 05	Funcionando
Cadastro e login de usuário	06 e 07	Funcionando
Bloqueio por perfil e sem token	08 e 09	Bloqueios corretos
Login admin e consultas autenticadas	10 a 12	Funcionando
Cadastro do produto (CREATE)	13	Criado, ID 6
Consulta do produto (READ)	14	Encontrado
Edição do produto (UPDATE)	15 e 16	Alterações salvas
Busca pelo nome	17	Encontrado
Exclusão do produto (DELETE)	18 e 19	Removido; depois 404

O CRUD na prática

Criei a *camisa do baiao* pelo `POST`, consultei pelo `GET`, mudei o preço e o estoque com `PUT` e excluí com `DELETE`. Depois de excluir, fiz mais um GET e recebi 404, porque aquele ID já não existia no catálogo.

O que aprendi com os testes

Entendi melhor a diferença entre pedir dados e mandar dados para o servidor. Também vi a importância dos códigos HTTP. Nem toda resposta vermelha é falha da API: nos testes de segurança, o servidor precisava barrar ações indevidas.

Outra parte importante foi a autenticação. Um cliente pode estar logado e mesmo assim não ter autorização para cadastrar produtos. Já o administrador consegue fazer alterações porque usa um token associado ao perfil correto.

Testes adicionais da coleção

A coleção também tem dois testes extras: um com paginação inválida (página 0 e limite 101) e outro acessando uma rota que não existe. Eles foram preparados para conferir respostas de erro. Não coloquei esses dois como executados porque não tenho imagens dos resultados deles neste material.

5 Conclusão

Com os prints e as respostas do Postman, consegui mostrar que a Artisan Store API está atendendo às rotas principais, consultando o SQLite e respeitando permissões de usuário. A parte que mais ajudou a entender foi testar o mesmo produto do cadastro até a exclusão, porque deu para acompanhar a mudança dos dados em cada etapa.

Esse trabalho também mostrou que testar API não é apenas procurar HTTP 200: tem que conferir o dado que voltou, a permissão usada e se o comportamento era o esperado em cada situação.

Dificuldades que apareceram

Durante a preparação, tive problemas para conectar o Postman ao servidor. A mensagem era de conexão recusada na porta 3000. Também apareceu um erro de chave estrangeira na sincronização do SQLite. Fui verificando pelo terminal até a API voltar a responder.

Outro ponto foi o acesso de administrador: o cadastro inicial dele dependia da situação do banco, então precisei conferir os usuários cadastrados e ajustar a conta local de testes. Depois disso, o login funcionou e consegui seguir com o CRUD.

Como eu repetiria essa demonstração

Primeiro iniciaria a API com `npm start` e verificaria a rota de saúde. Depois faria o login de administrador, cadastraria um produto novo e guardaria o ID retornado. Com esse mesmo ID, faria a consulta, a edição e a exclusão. No final, uma nova consulta deveria responder que o produto não foi encontrado.

Essa sequência foi a mais clara para mostrar o funcionamento do backend na prática.

Observação: as capturas são originais e mostram os tokens JWT de teste, conforme utilizados durante a demonstração. O material é voltado à entrega ao professor e não deve ser publicado em acesso aberto enquanto essas credenciais estiverem válidas.